

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación
(A Distancia)

Programación 1

Trabajo Práctico Integrador
*Sistema de Gestión de Datos de
Países*

Alumno: Fernando Muiños

Tutora: Guadalupe Maricchiolo

17 de junio de 2026

Índice

Marco Teórico.....	3
1. Estructuras de Datos: Listas y Diccionarios	3
2. Modularización y Funciones	3
3. Estructuras de Control de Flujo	3
4. Algoritmos de ordenamiento	4
5. Generación de Estadísticas.....	4
6. Persistencia de datos y manejo de archivos (CSV)	4
Arquitectura y diseño del sistema	5
Lógica general del programa	5
Diagramas de cada módulo del menú	6
Interfaz de usuario y pruebas de funcionamiento	12
Dificultades y conclusiones.....	13
Conclusión.....	13
Enlaces del proyecto	13
Bibliografía	14

Marco Teórico

El desarrollo del presente Trabajo Práctico Integrador se fundamenta en la aplicación práctica de diversos paradigmas y herramientas provistas por el lenguaje de programación Python. A continuación, se detallan los conceptos técnicos centrales utilizados para la construcción del sistema:

1. Estructuras de Datos: Listas y Diccionarios

Para el almacenamiento temporal y la manipulación de los datos en memoria durante la ejecución del programa, se optó por una estructura combinada de listas y diccionarios:

- **Listas:** Son colecciones ordenadas y mutables de elementos. En este proyecto, se utilizó una lista principal (*lista_paises*) para contener la totalidad de los registros leídos desde el archivo de texto, permitiendo iterar sobre ellos mediante bucles *for*.
- **Diccionarios:** Son colecciones no ordenadas de datos estructurados en pares de "clave-valor" (*key-value*). Cada país en el sistema fue modelado como un diccionario (ej. `{'nombre': 'Argentina', 'poblacion': 45376763}`), lo que facilita el acceso directo a atributos específicos de cada registro sin depender de índices numéricos, mejorando la legibilidad del código.

2. Modularización y Funciones

Siguiendo el principio de diseño de software de "responsabilidad única", el sistema fue segmentado en funciones independientes. Cada operación principal (agregar, actualizar, buscar, filtrar, ordenar y calcular estadísticas) fue encapsulada en su propia función. Esto no solo facilita la lectura y el mantenimiento del código, sino que aísla los posibles errores de ejecución.

3. Estructuras de Control de Flujo

El enrutamiento de la lógica del programa se logró mediante condicionales y bucles:

- **Condicionales (*if/elif/else*):** Utilizados para el menú principal y la bifurcación de los filtros, permitiendo que el programa tome decisiones basadas en la entrada del usuario.

- **Bucles (*while* y *for*):** El bucle *while True* garantizó la persistencia del menú principal y la validación iterativa de datos (evitando el colapso del sistema por ingresos erróneos). Los bucles *for* fueron esenciales para recorrer la lista de diccionarios en las operaciones de búsqueda y filtrado.

4. Algoritmos de ordenamiento

Para el ordenamiento de los registros, se aprovechó la función nativa *sorted()* de Python, la cual devuelve una nueva lista ordenada sin mutar la original.

5. Generación de Estadísticas

El cálculo de indicadores se resolvió mediante funciones nativas de agregación de Python, optimizando el rendimiento:

- Se utilizaron *max()* y *min()* para encontrar los extremos poblacionales.
- Se empleó la función *sum()* para totalizar variables numéricas y calcular promedios.
- Se aplicaron diccionarios dinámicos para agrupar y contabilizar las ocurrencias categóricas (cantidad de países por continente).

6. Persistencia de datos y manejo de archivos (CSV)

Para garantizar que la información perdure entre distintas ejecuciones del programa, se implementó el módulo nativo *csv*. Se utilizó *csv.DictReader* para la carga inicial de los datos, transformando cada fila del archivo de texto plano en un diccionario. Para el guardado de nuevos registros y actualizaciones, se empleó *csv.DictWriter* abriendo el archivo en modo escritura (*mode='w'*), logrando así un sistema con persistencia bidireccional.

Arquitectura y diseño del sistema

Lógica general del programa

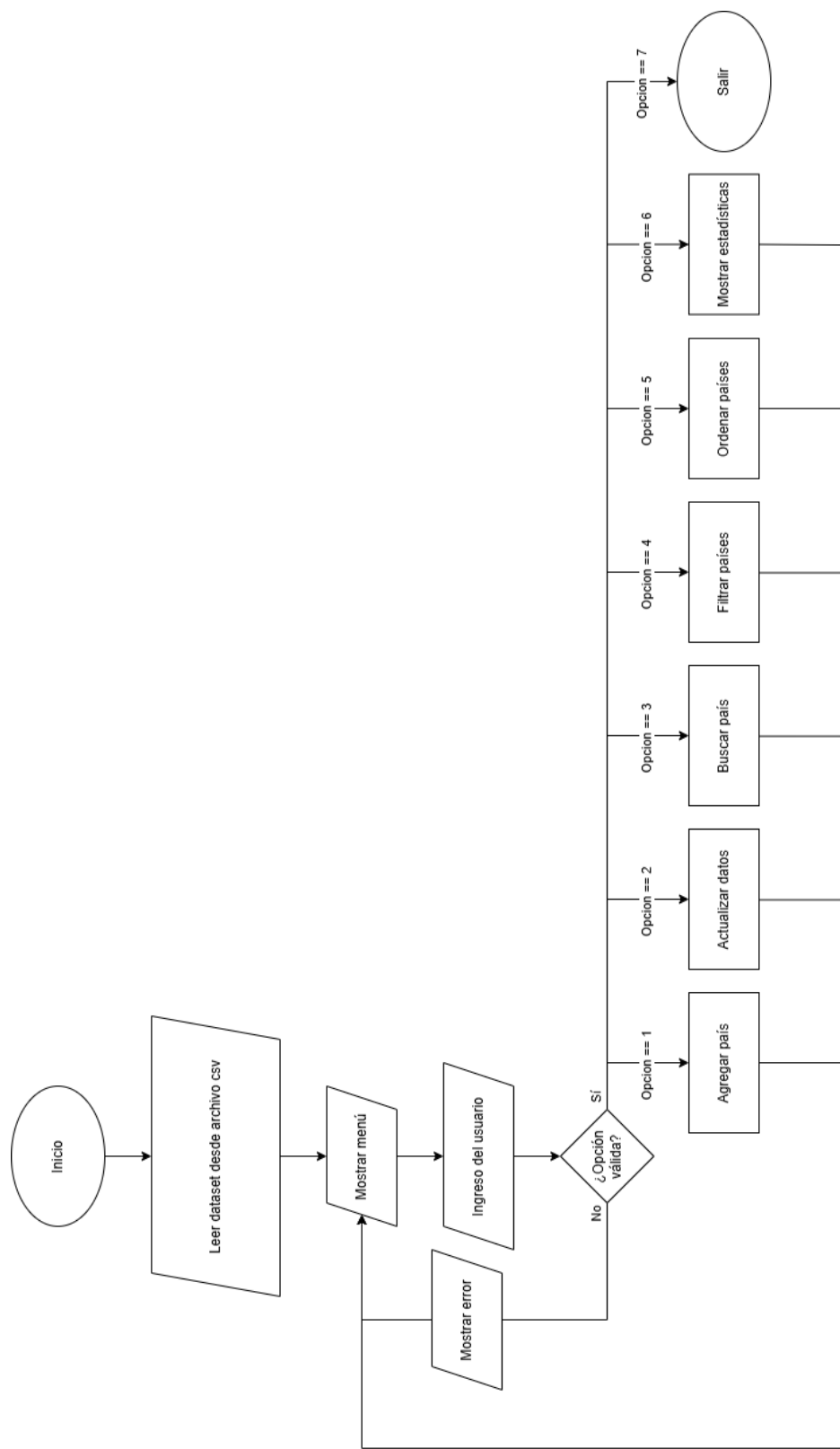


Figura 1. Diagrama general del sistema.

Diagramas de cada módulo del menú

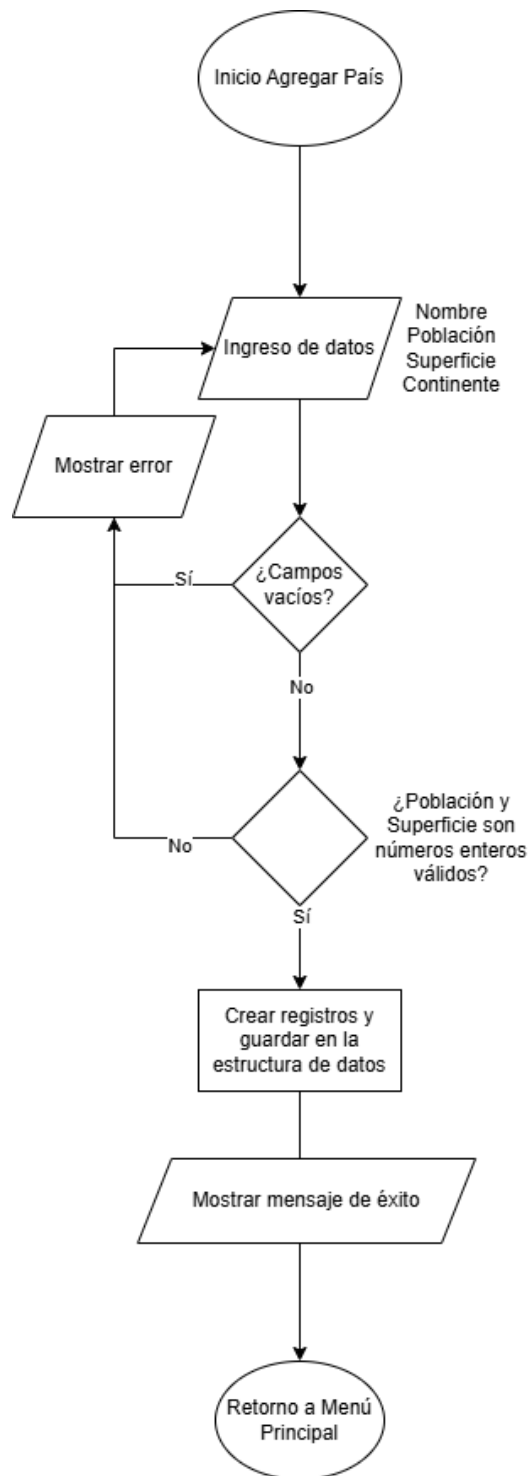


Figura 2. Diagrama de "Agregar País"

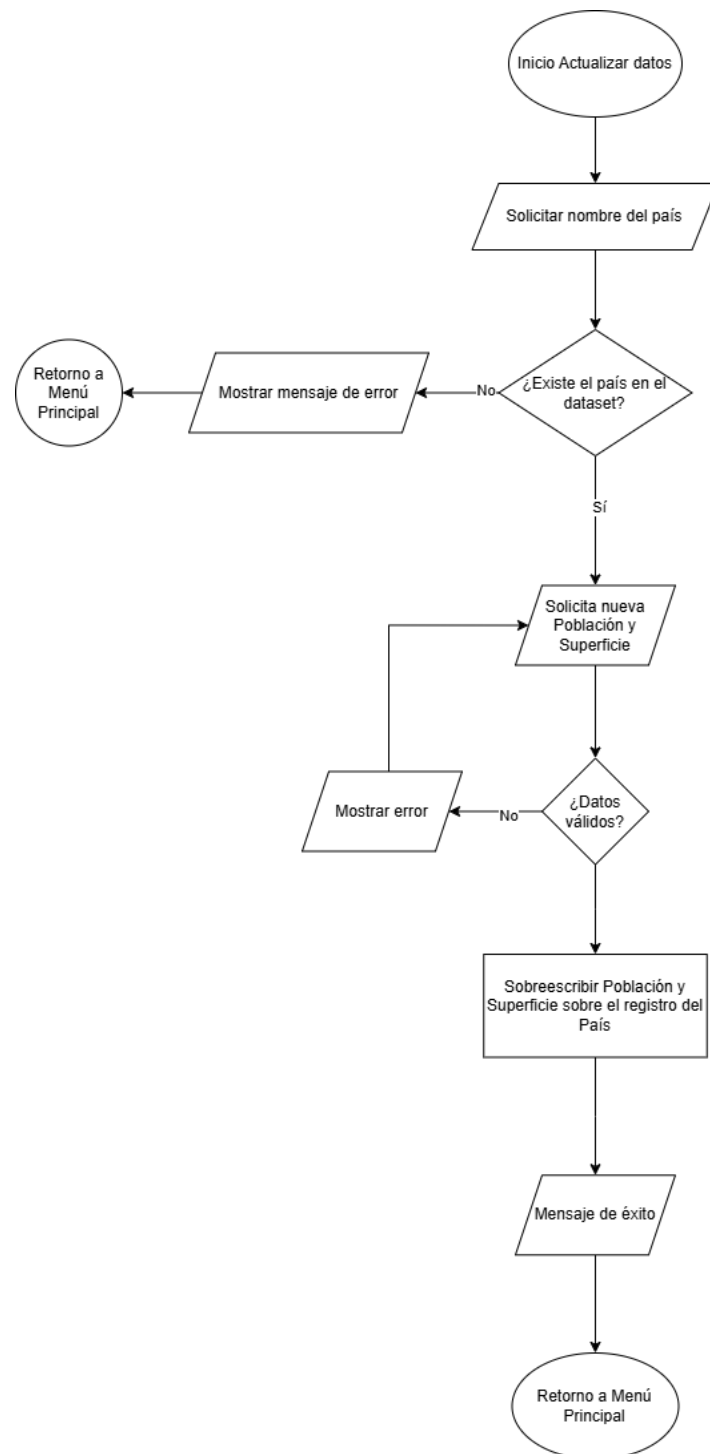


Figura 3. Diagrama de actualizar datos.

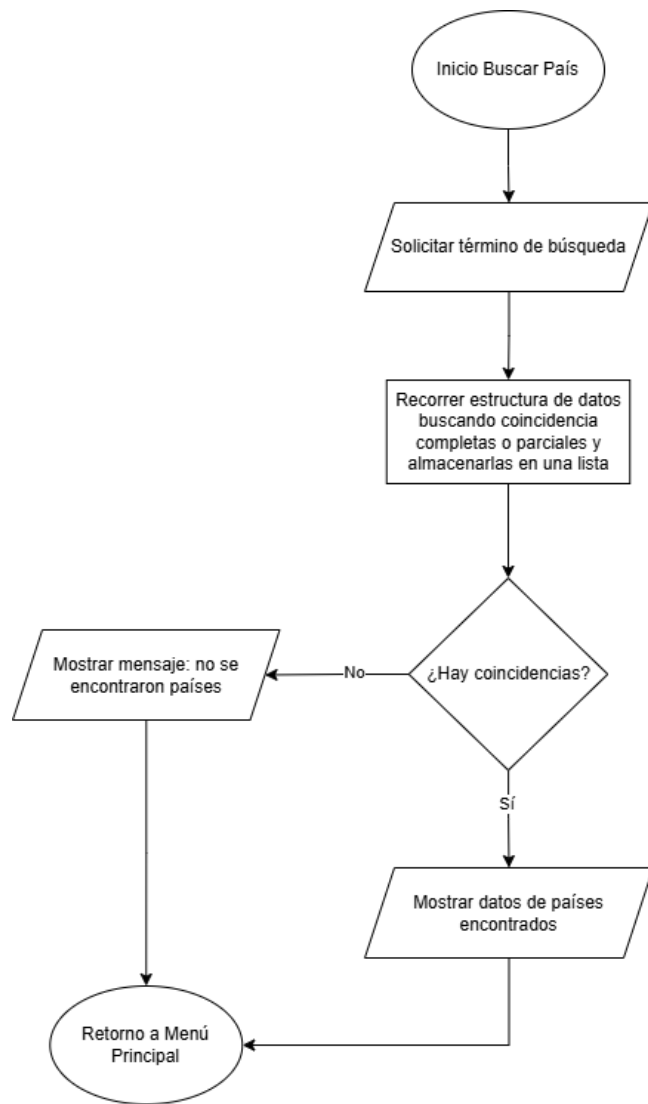


Figura 4. Diagrama de "Buscar País".

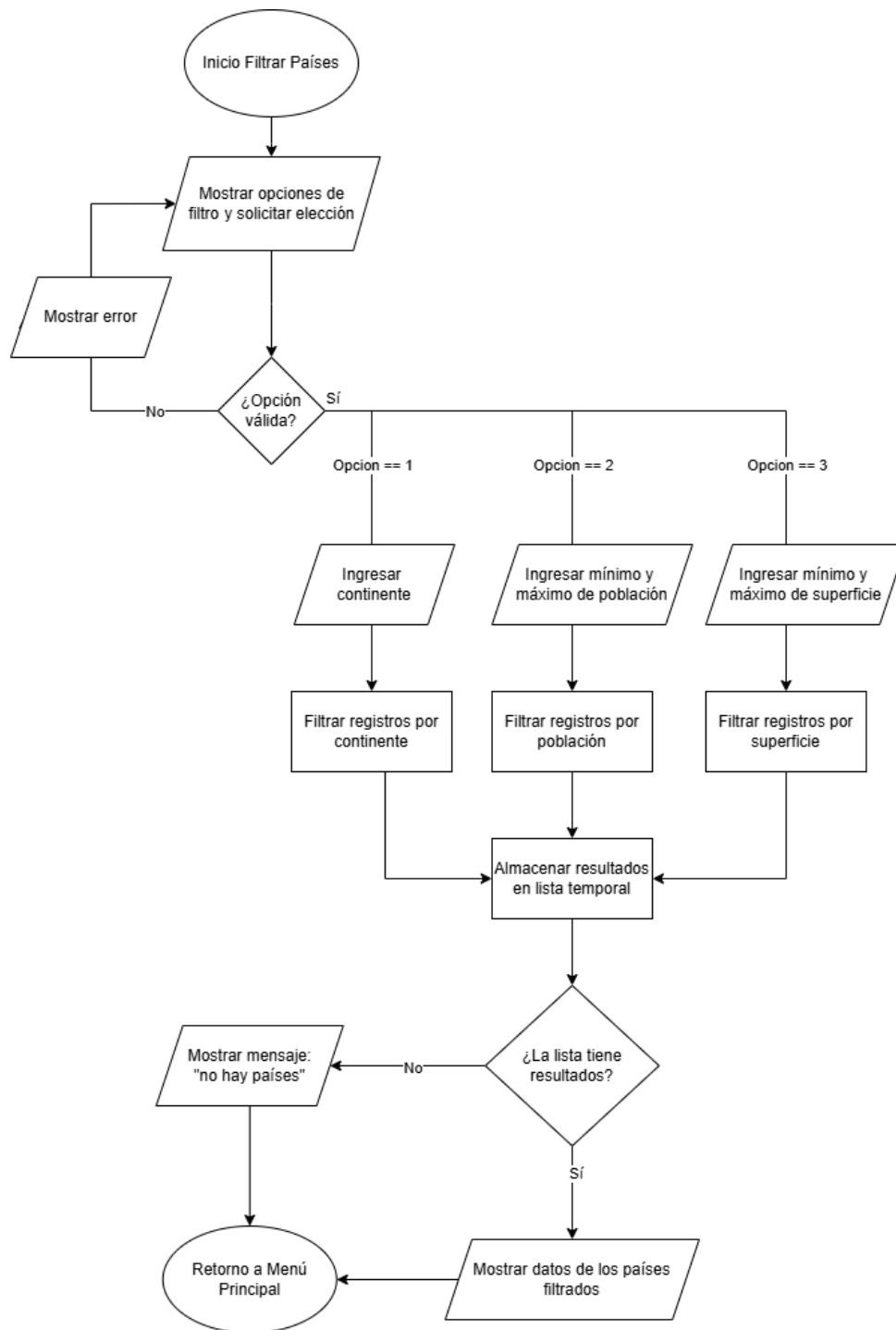


Figura 5. Diagrama de "Filtrar Países".

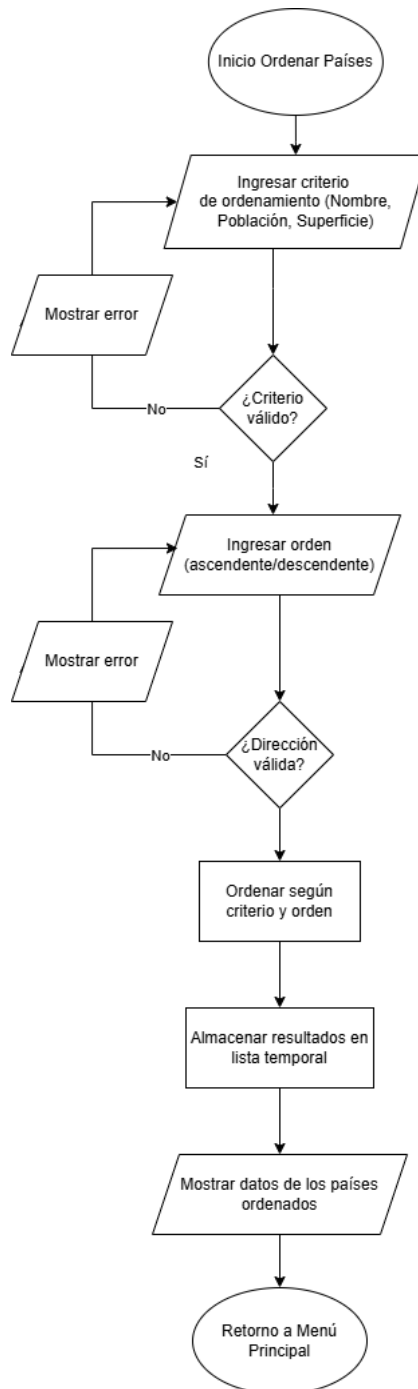


Figura 6. Diagrama de "Ordenar Países".

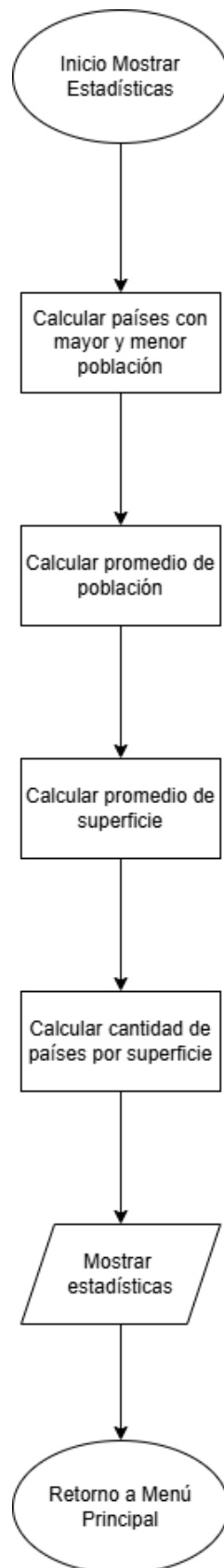


Figura 7. Diagrama de "Mostrar Estadísticas".

Interfaz de usuario y pruebas de funcionamiento

```
Datos cargados correctamente.

--- GESTIÓN DE DATOS DE PAÍSES ---
1. Agregar un país
2. Actualizar datos de un país
3. Buscar un país
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
Ingrese una opción (1-7): |
```

Figura 8. Menú principal al abrir el programa.

```
Ingrese una opción (1-7): 3

--- BUSCAR PAÍS ---
Ingrese el nombre o fragmento a buscar: ar

Se encontraron 1 coincidencia(s):
-----
Nombre: Argentina
Población: 45376763
Superficie: 2780400 km2
Continente: América
-----
```

Figura 9. Búsqueda con resultados (ej. al buscar “ar”)

```
Ingrese una opción (1-7): 6

--- ESTADÍSTICAS DE LOS PAÍSES ---
• País con MAYOR población: brasil (213,993,437 hab.)
• País con MENOR población: azerbaiyán (10,412,000 hab.)
• Promedio de población global: 76,033,150 hab.
• Promedio de superficie global: 3,252,283.50 km2

• Cantidad de países por continente:
  - América: 3 país(es)
  - Asia: 2 país(es)
  - Europa: 2 país(es)
  - Oceanía: 1 país(es)
  - África: 2 país(es)
```

Figura 10. Cuadro de estadísticas.

```
Ingrese una opción (1-7): 4
--- FILTRAR PAÍSES ---
1. Por Continente
2. Por Rango de Población
3. Por Rango de Superficie
Elija un filtro (1-3): 2
Ingrese la población mínima: a
Error: Los rangos deben ser números enteros válidos.
```

Figura 11. Manejo de errores.

Dificultades y conclusiones

Durante el desarrollo del Trabajo Práctico, me encontré con ciertos desafíos que requirieron investigación y análisis para su correcta implementación:

- **Manejo de referencias y mutabilidad:** Al desarrollar el módulo de actualización de datos, surgió la pregunta sobre cómo impactan los cambios en la base de datos principal al modificar una variable temporal (*pais_encontrado*). Se entendió que, en Python, las variables que contienen diccionarios actúan como referencias al mismo objeto en memoria. Entender ese concepto permitió actualizar los datos directamente sin necesidad de buscar índices o reinsertar elementos en la lista.
- **Persistencia bidireccional:** Asegurar que el archivo .csv no solo se leyera al inicio, sino que se sobrescribiera correctamente tras cada registro agregado o modificación, requirió entender los modos de apertura de archivos (*mode='w'*) y la escritura mediante *csv.DictWriter*.

Conclusión

La realización de este proyecto significó una consolidación fundamental de los conceptos teóricos de Programación 1. Se pudo transformar el planteo diseñado en los diagramas de flujo hacia la codificación de un sistema robusto, modular y persistente. La experiencia de planificar la arquitectura antes de codificar demostró ser una práctica que reduce el margen de error y permite desarrollar software funcional.

Enlaces del proyecto

Repositorio de GitHub: <https://github.com/fdmuinos/UTN-TUPaD-Programacion1-TPI>

Video de demostración: <https://drive.google.com/file/d/1eN0VxJuq-zyu1RpFFilG0wYfWhHab95I/view?usp=sharing>

Bibliografía

Python Software Foundation (2026). Python 3.x Documentation. Recuperado de:
<https://docs.python.org/3/>